Deliverable I: Requirements Engineering

Due: September 26, 2025

This is your team's **first project submission**. Think of it as a **"mini blueprint"** of your system: what it will do, who needs it, and how you will capture requirements.

1. Project Overview

- Write **2–3 paragraphs** describing your project idea.

Answer:

The project we are developing is a class registration system. The system will be designed for three roles: professors, students, and admins. This system will allow professors to post and manage their courses, as well as add information such as meeting times and syllabi. For students, this system will allow them to search for classes and add/drop classes as required to build a schedule. The administrators will be able to override and perform the functions for both the students and professors if needed. They will also be able to configure settings such as adding/dropping deadlines, or changing the maximum credit hours a student can take in a given semester.

The goal is to develop a system similar to the one that we all use to register for classes each semester. We will focus on two degree plans (CS and CE) to minimize the amount of data we will need to test the functionality of the system. In addition, if the system is operating as intended, we plan to include an AI-powered course recommendation feature. This system will take into account classes students have completed as well as classes that should be focused on earlier rather than later due to prerequisites.

- State whether you will use **Agile** or **Plan-driven** development.

Answer: Our team plans on using an agile development approach.

- Explain in **1–2 sentences** why you chose that approach.

Answer:

The system is not critical or life-dependent, and the timeline for this approach seems to be the best fit. We plan to organize the work into 1 or 2-week sprints, focusing on the deliverables and conducting testing along the way. Since we have a smaller timeline, we thought it would be best to use this approach so that we can start small with our project and add more features later if we have the time.

2. Stakeholders

· Make a list of all the people/groups who care about your system (e.g., end users, managers, owners, outside organizations).

· For each stakeholder:

  - **Name/role**

  - **What they want** from the system

Answer:

- Professors
  - Role: End Users
  - What they want:
    - Ability to post courses they are teaching each semester, along with details that are important to students who would want to register for their class (meeting time, seats, etc.)
    - Ability to post a syllabus for their course
    - A way to edit information in their posts (meeting room, office hours, etc.)
- Students
  - Role: End Users
  - What they want:
    - The ability to search through and filter courses offered
    - Ability to add and drop classes and view the available seats
    - Receive notifications when they are moved off a waitlist into the class
    - Explanations if unavailable to register (prerequisites not met, schedule conflict, etc.)
- Administrators
  - Role: System Maintenance/ Managers
  - What they want:
    - Override functions if the system is not operating properly
    - Account management (create a new account, delete an account, grant privileges to new admin, student, or professor accounts)
    - Create start dates and deadlines for when registration opens/ closes

3. Requirements Elicitation

· Explain how you will **gather requirements** (e.g., interviews, surveys, brainstorming).

· Create a **draft questionnaire** with at least 5 sample questions you would ask.

· Show how you will **organize requirements** later (for example: “interface,” “functionality,” “security”).

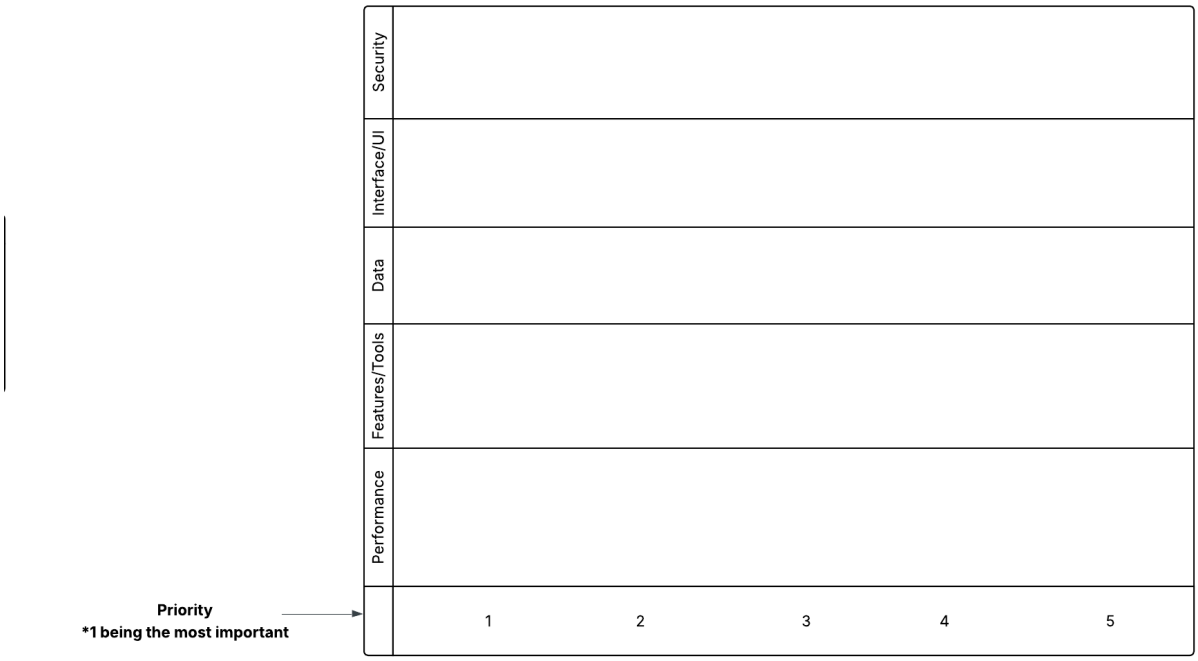
Answer:

The end users and people who will ultimately benefit from this system will be students. We could choose to interview/survey students on campus, but since we are already students who have experience registering for classes with UTD's registration system, it would be more efficient to brainstorm.

Questionnaire/questions to think about when brainstorming:

1. What features does UTD's registration system offer that are absolutely necessary in a registration system?
2. What problems/issues does UTD's current registration system have and how can we fix or improve them?
3. Do we feel lost or confused when registering for classes using UTD's current registration system?
4. As an Admin, what features/tools are most beneficial to make the workflow easier and faster?
5. How confident are you as a student when knowing what classes you can take next?

Organize Requirements:



We will organize our requirements into each subcategory and rate them using a tier list with 1 having highest priority and 5 having the smallest.

4. Requirements Specification

1. **Write functional requirements** (what the system does). Use the format:

    a. "The system shall …"

    b. Example: *"The system shall allow users to reset their password via email."*

    c. Write a complete set of **functional requirements**.

2. **Write non-functional requirements** (quality or constraints, like speed, security, reliability).

    a. Example: *"The system shall respond to user actions within 2 seconds."*

    b. Write at least **2–3 non-functional requirements**.

    c. Each non-functional requirement should connect to at least one functional requirement.

3. **Show requirements in two formats**:

    a. One text-based (natural language, structured, or tabular).

    b. One diagram (UML use case or sequence diagram).

Answer:

Functional Requirements (Natural Language)

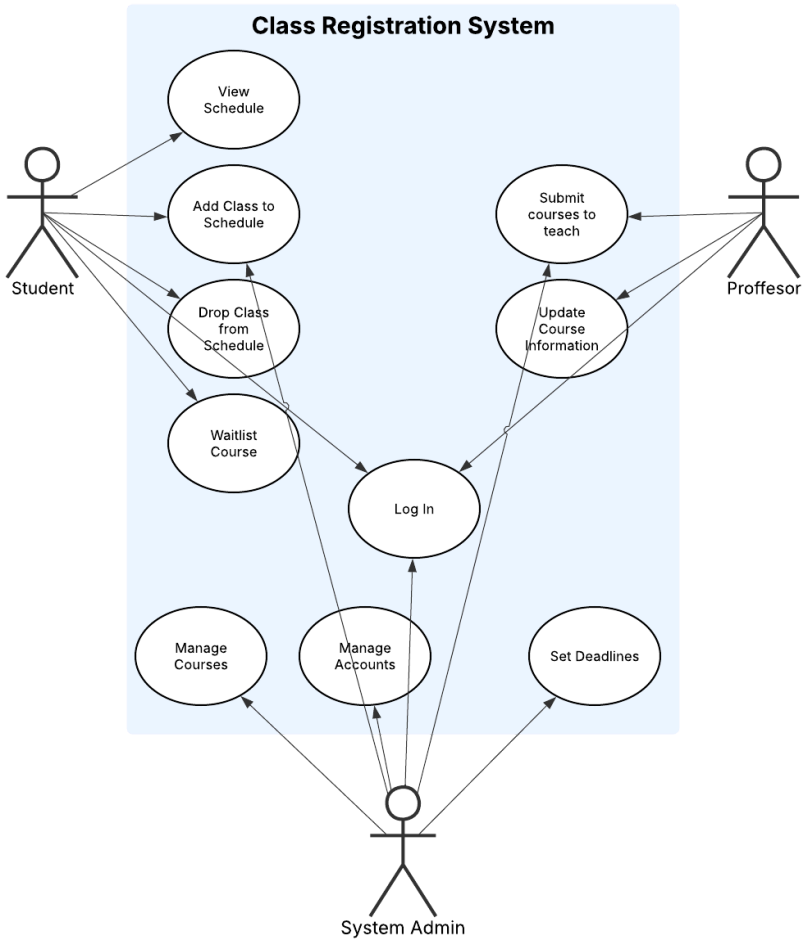
1. The system shall allow all users to log in using their unique username and password
2. The system shall allow professors to submit what courses they are going to teach
3. The system shall allow professors to provide information about their courses, including the meeting date and times, course description, syllabus, and how many students can take the course
4. The system shall keep track of each student, as well as their major and what courses they have already taken
5. The system shall allow students to search through all provided courses and choose to add or drop classes to their schedule
6. The system shall not allow students to register for a course if they have not met the corequisites or prerequisites
7. The system shall not allow students to register for a course if they have already registered for the maximum number of credit hours
8. The system shall not allow students to register for courses that meet at the same time
9. The system shall not allow students to register for a course that is full

10. The system shall allow students to waitlist a course and notify them once there is a spot available
11. The system shall not allow students to register before their designated time
12. The system shall not allow students to drop classes after the designated deadline
13. The system shall allow system admins to submit courses on behalf of professors
14. The system shall allow system admins to add or drop courses from students' schedules
15. The system shall allow system admins to set the designated registration times of each student, as well as the final day to drop classes
16. The system shall allow system admins to set the maximum number of credit hours that a student can register for

Non-Functional Requirements (Natural Language)

1. The system shall have users log in using their school-issued ID as their username, as well as a generated password given to them
2. The system shall at least be readily available to students at all hours from their designated registration time to the final day to drop classes 99% of the time
3. The system shall allow users to log in within 10 seconds of entering their username and password
4. The system shall only allow certain users to view students' personal data as specified by FERPA

UML Use Case Diagram



5. Conflict Resolution (if needed)

- If two requirements clash (e.g., “system must be cheap” vs. “system must be ultra-secure”), explain the conflict and how your team would handle it.

Answer:

Most of our functional and non-functional requirements do not conflict with each other. However, if in the future we were to find conflicting design choices, we will handle it by prioritizing what is most important. In our system, we will prioritize all requirements that stem from security, reliability, and availability. Because registering for classes is time sensitive, we want to prioritize availability above all else. Also, we want to make sure that once a student registers for a course or a professor adds a course to the course list, it is done. If our system is not reliable to accomplish the tasks being asked, this could leave a student without an important class in their schedule that they need to graduate. Another one of our priorities is security. We want to reassure students that no other student can manipulate their class schedule. We also want to make sure that no one can see a student's sensitive information, which may go against FERPA. If any of these requirements were to conflict, then our team would come together and weigh the options to see which one we think is more important for our users.

6. Requirements Validation

- Describe how you will **check that your requirements are correct** (e.g., reviews, prototyping, writing test cases).
- For each requirement set, comment on:
    - **Verifiability** – can you test it?
    - **Comprehensibility** – is it clear?
    - **Traceability** – do you know where it came from?
    - **Adaptability** – can it be changed later?

Functional Requirements (Natural Language)

1. The system shall allow all users to log in using their unique username and password
    a. Verifiability: Yes, attempt to log in.
    b. Comprehensibility: Yes.
    c. Traceability: Yes.
    d. Adaptability: Yes, for example, we could change the unique username to an email.
2. The system shall allow professors to submit what courses they are going to teach
    a. Verifiability: Yes, log in to the professor portal and add a course.
    b. Comprehensibility: Yes.
    c. Traceability: Yes.
    d. Adaptability: Yes, for example, we could change courses to sections.
3. The system shall allow professors to provide information about their courses, including the meeting date and times, course description, syllabus, and how many students can take the course
    a. Verifiability: Yes, the class should be populated with this information.
    b. Comprehensibility: Yes.
    c. Traceability: Yes.
    d. Adaptability: Yes, we could add more attributes.
4. The system shall keep track of each student, as well as their major and what courses they have already taken
    a. Verifiability: Yes, check that the values associated with a student are accurate.
    b. Comprehensibility: Yes.
    c. Traceability: Yes.
    d. Adaptability: Yes, for example, keeping track of professors can be added.
5. The system shall allow students to search through all provided courses and choose to add or drop classes to their schedule
    a. Verifiability: Test that students can search courses, add them and drop them. Reflecting accurate changes.
    b. Comprehensibility:The requirement clearly states that students can search and modify schedules.
    c. Traceability:It links to use cases for browsing courses and schedule management

    d. Adaptability:It allows flexibility for new course attributes or UI changes without altering core functionality.
6. The system shall not allow students to register for a course if they have not met the corequisites or prerequisites
    a. Verifiability:Test that the system blocks registration for when prerequisites or corequisites are not met.
    b. Comprehensibility:The requirement clearly mandates the registration rules.
    c. Traceability:It traces use cases for eligibility checks, linking to course data, student records, and validation logic.
    d. Adaptability:This requirement supports changes like the validation method without altering core functionality.
7. The system shall not allow students to register for a course if they have already registered for the maximum number of credit hours
    a. Verifiability: Yes, deny the request and display a message if it has reached the max.
    b. Comprehensibility: Yes.
    c. Traceability: Yes.
    d. Adaptability: Yes, for example, swapping can be allowed.
8. The system shall not allow students to register for courses that meet at the same time
    a. Verifiability: Yes, deny the request and display a message if there is overlap.
    b. Comprehensibility: Yes.
    c. Traceability: Yes.
    d. Adaptability: No, there is nothing to change.
9. The system shall not allow students to register for a course that is full
    a. Verifiability: Yes, deny the request and display a message if it has reached the max.
    b. Comprehensibility: Yes.
    c. Traceability: Yes.
    d. Adaptability: No, there is nothing to change.
10. The system shall allow students to waitlist a course and notify them once there is a spot available
    a. Verifiability: Yes, we can queue and dequeue students from the waitlist.
    b. Comprehensibility: Yes.
    c. Traceability: Yes.
    d. Adaptability: Yes, for example, we could change the notification to be an auto-register.
11. The system shall not allow students to register before their designated time
    a. Verifiability:Test that the system blocks registration from a student if they arent registering in an allotted time slot.
    b. Comprehensibility:the requirement clearly states students can only register at a certain time
    c. Traceability:link to use cases for time-based registration access, mappable to student schedules and validation logic.
    d. Adaptability:Supports changes to time slot rule changes and registration rules changes without altering the core functionality
12. The system shall not allow students to drop classes after the designated deadline
    a. Verifiability:Test that the system blocks attempts to drop classes after the deadline
    b. Comprehensibility:the requirement clearly states students cant drop classes post-deadline.

    c. Traceability:mappable to student schedules, deadline data, validation logic
    d. Adaptability:supports changes in deadline dates or exception policies without altering core functionality
13. The system shall allow system admins to submit courses on behalf of professors
    a. Verifiability:Test that admins can submit course details for professors
    b. Comprehensibility:clearly states the purpose of having an interface that admins can utilize
    c. Traceability:mappable to course catalog, admin interface, and database.
    d. Adaptability:supports change in course attributes without altering core functionality.
14. The system shall allow system admins to add or drop courses from students' schedules
    a. Verifiability:test that admins can add or drop courses from a student's schedule with changes accurately reflected in the students record
    b. Comprehensibility:clearly states admins can modify students class schedules
    c. Traceability:mappable to student records, course data, and admin interface
    d. Adaptability:flexible system that allows changes to admin controls without altering core functionality.
15. The system shall allow system admins to set the designated registration times of each student, as well as the final day to drop classes
    a. Verifiability: Yes, students should not be able to register before their designated time, and cannot drop them after the final day.
    b. Comprehensibility: Yes.
    c. Traceability: Yes.
    d. Adaptability: Yes, for example, the condition to be unable to drop classes could be changed to not receiving a refund.
16. The system shall allow system admins to set the maximum number of credit hours that a student can register for
    a. Verifiability: Yes, the value for the maximum number of credit hours should be updated.
    b. Comprehensibility: No, it’s ambiguous, as it could mean the admins can make one student have a different maximum from the rest, or if the change will always apply to all students.
    c. Traceability: Yes.
    d. Adaptability: Yes, for example, to clear up the ambiguity.

Non-Functional Requirements (Natural Language)

1. The system shall have users log in using their school-issued ID as their username, as well as a generated password given to them
    a. Verifiability:test that users can log in using their school-issued ID as the username and password with successful authentication.
    b. Comprehensibility:clearly specifies that issued usernames and passwords will be used for authentication to use system
    c. Traceability:maps to login interface, user database
    d. Adaptability:supports changes in additional authentication methods without altering core functionality
2. The system shall at least be readily available to students at all hours from their designated registration time to the final day to drop classes 99% of the time

    a. Verifiability:test system uptime by monitoring 24/7 ensuring it meets 99% uptime.
    b. Comprehensibility:clearly mandates a 99% uptime requirement
    c. Traceability:traces to performance metrics and server infrastructure
    d. Adaptability:allows for change in infrastructure without changing the main requirement
3. The system shall allow users to log in within 10 seconds of entering their username and password
    a. Verifiability:test the system authenticates within 10 seconds of submitting valid credentials
    b. Comprehensibility:clearly states a 10 second log-in time
    c. Traceability:mappable to authentication logic, server response, and metrics.
    d. Adaptability:supports change in authentication methods without altering core requirement
4. The system shall only allow certain users to view students' personal data as specified by FERPA
    a. Verifiability: Yes, we can use certain user accounts and try to access other students' personal data.
    b. Comprehensibility: No, "certain users" is vague.
    c. Traceability: Yes.
    d. Adaptability: Yes, for example, to clarify who these "certain users" are.